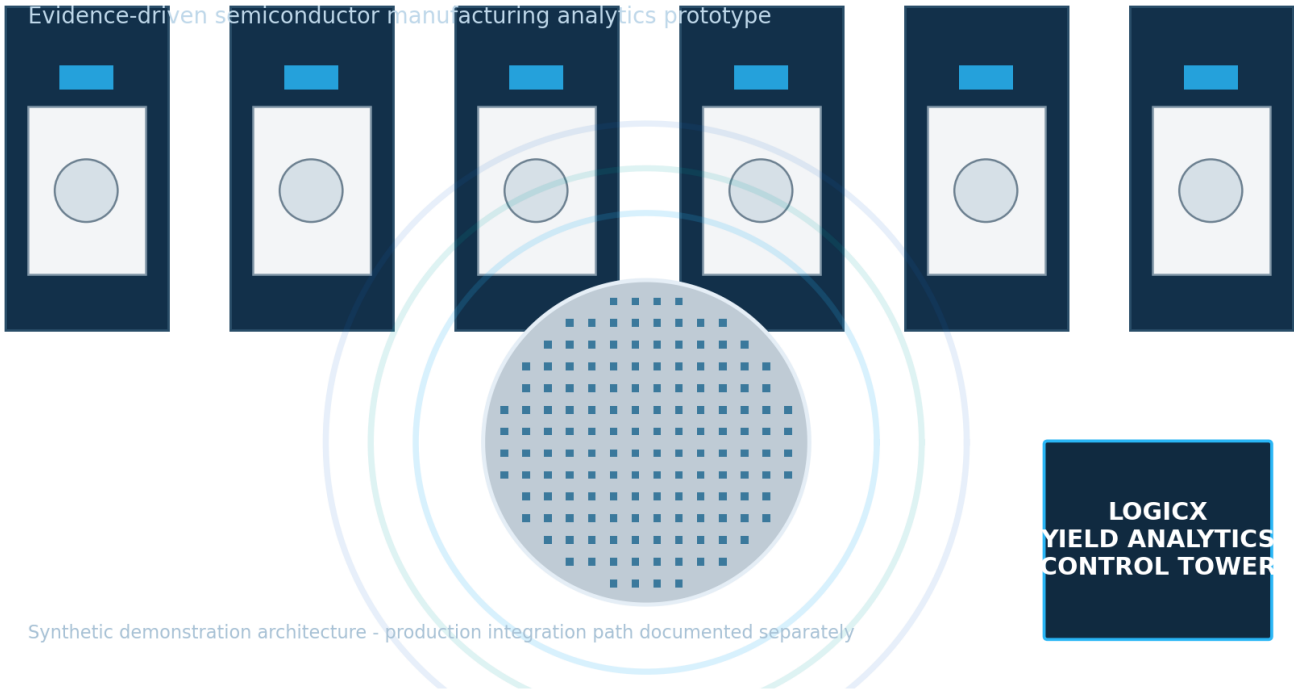




WAFER YIELD ROOT CAUSE & DEFECT PATTERN ANALYSER

Complete Technical, Industry and Prototype Dossier

Team LogicX | IBM Bobathon | Challenge S1

Document purpose

Enable a technical judge, semiconductor engineer, or product reviewer to understand the problem, the complete prototype workflow, why each analytics signal exists, what evidence is produced, how the system is governed, and how the prototype maps to a real fab deployment.

Field	Value
Project	Wafer Yield Root Cause & Defect Pattern Analyser
Prototype	LogicX semiconductor manufacturing analytics control tower
Repository	github.com/ronaksavsani181/bob-ai-hackathon-LogicX
Data mode	Deterministic synthetic demonstration data; seed = 42
Current stack	React 18, TypeScript, Vite, Tailwind, FastAPI, SQLAlchemy, PostgreSQL 15, pandas, NumPy, SciPy, scikit-learn, Docker
Prepared	September 2026
Document status	Judge-ready technical dossier; production integration requirements included

Scope, Source Coverage and Important Data Boundary

What is documented, what is verifiable from the prototype, and what is intentionally not claimed

This dossier consolidates the technical architecture and operating logic of the LogicX prototype using the public project repository, its extended technical documents, and semiconductor-industry references. The repository is the primary implementation source of truth for the prototype architecture and algorithms.

Important prototype boundary

The repository states that all asset footprints, metrics, wafer tracking points and parameters are synthetic, deterministic demonstration records generated with seed=42. They do not represent the operational metrics, intellectual property, yields or equipment status of an operating fab. This document therefore distinguishes prototype behavior from production validation requirements.

The supplied shared ChatGPT conversation link was reachable as a share page but its transcript was not exposed as machine-readable content to the retrieval interface. To avoid inventing inaccessible details, this dossier uses the repository and the technical context already available in the conversation as the evidence base.

What the judge should understand

- LogicX is not a chatbot. The core execution path is deterministic analytics and evidence fusion.
- The system moves from fleet monitoring to lot investigation, wafer spatial analysis, evidence-based root-cause candidates, pre-run risk and governed corrective-action review.
- The prototype is designed around traceability: evidence IDs, algorithm versions, timestamps and reviewer approval are first-class concepts.
- The production story is intentionally explicit: real fab connectors, calibrated thresholds, historical replay, cybersecurity and controlled write-back are future integration stages rather than claims about the current demo.

Executive Summary

The complete story in one view

Judge Demo Storyboard: One Yield Excursion, End to End



The product is not a chatbot: the core value is a repeatable engineering workflow that turns heterogeneous fab evidence into a reviewable hypothesis set.

Outcome: faster triage, clearer evidence, earlier intervention, explicit accountability

Figure 1. End-to-end judge narrative reconstructed from the implemented workflow.

Core problem

A semiconductor fab can observe an excursion in yield long after multiple process steps, chambers, maintenance events, sensor traces, metrology records and defect observations have accumulated. The engineering challenge is not merely detecting that yield changed; it is correlating heterogeneous evidence, understanding spatial signatures, determining whether a chamber or process parameter repeatedly appears in similar excursions, and deciding what should be investigated next.

LogicX response

LogicX converts this into a repeatable control-tower workflow. A lot is selected, historical context is loaded, yield deviation and parameter anomalies are calculated, wafer defects are classified spatially, chamber recurrence and maintenance proximity are evaluated, similar historical lots are retrieved, and seven signals are fused using fixed weights into a small set of evidence-supported root-cause candidates. A separate pre-run model evaluates upcoming lots using only information that was available before the planned start time.

Why the architecture is credible as a prototype

Dimension	Implemented prototype behavior
Determinism	Root-cause ranking uses fixed formulas and a controlled cause vocabulary; no LLM is used in scoring.
Traceability	Results carry evidence_id, algorithm_version and generated_at.
Leakage protection	Pre-run features are hard-whitelisted and future-dated columns raise a hard error.
Human control	Corrective actions remain pending until a named engineer reviews them.
Separation of concerns	Frontend renders results; analytics remain in backend services.
Reproducibility	Synthetic data generation is deterministic with seed=42; tests cover analytics and guardrails.

Semiconductor Industry Context

Why yield learning requires multi-source, time-aligned evidence

Semiconductor Manufacturing Context: Where LogicX Fits

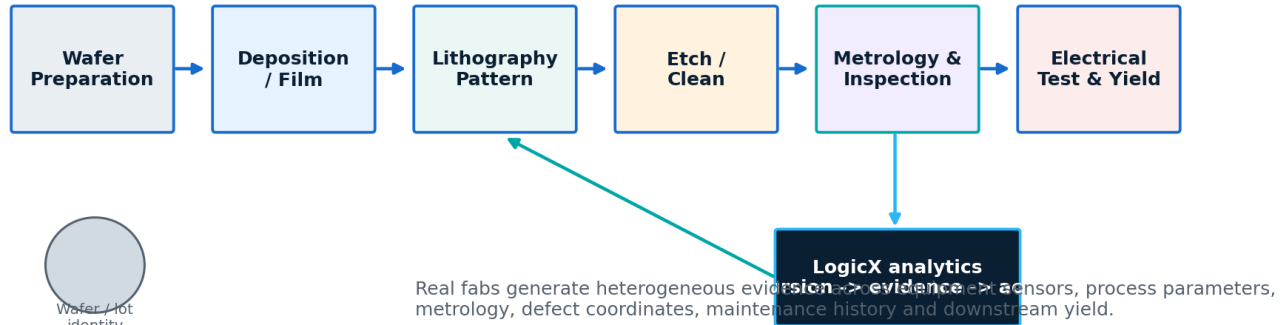


Figure 2. Simplified semiconductor manufacturing flow and the position of yield analytics.

Modern semiconductor manufacturing repeatedly moves wafers through patterning, deposition, etch, cleaning, metrology and inspection steps. The same wafer can accumulate evidence from many process tools and measurement systems. ASML describes optical metrology, e-beam inspection, in-scanner metrology and sensor data as part of the process-control environment; it also describes using algorithms and machine learning to interpret interactions among process factors that contribute to defects. [7][8]

KLA product descriptions similarly position wafer inspection around defect discovery, classification, monitoring and process-debug workflows, with optical, e-beam, sensor and AI-assisted inspection capabilities. [9]

For LogicX, these references are not claims that the prototype is connected to ASML, KLA or any production fab. They establish the industry context for why a system that joins lot identity, tool/chamber history, process parameters, metrology, defect coordinates, maintenance and yield is a realistic integration target.

Industry data streams that matter to LogicX

Fab evidence	What it can tell an engineer	LogicX representation
Lot / wafer identity	Which product, node and wafers were affected	lots, wafers
Process run context	Which recipe, chamber and tool processed the wafer	runs, recipes, chambers
FDC / trace signals	What equipment and process parameters did during the run	trace_data, process_parameters
Metrology	What CD, overlay, film or other measurements looked like	metrology
Inspection defects	Where defects occurred and their class	defects
Maintenance	Whether a preventive maintenance event preceded a shift	maintenance_events
Electrical yield	Which wafers / lots ultimately failed	yield_results

The Critical Engineering Problem

Why simple dashboards are not enough

1. Detection is easier than diagnosis

A chart can show that the fleet yield moved. The harder task is explaining whether the excursion is associated with one chamber, one process parameter, a spatial defect signature, a post-maintenance period, a recurring exposure-time effect, or a combination. LogicX therefore treats detection as the first gate, not the final answer.

2. Evidence lives in different time and identity domains

Lot metadata, tool events, process parameters, sensor traces, metrology, defects and maintenance records have different timestamps and granularities. A credible root-cause workflow must align them around stable identities such as lot, wafer, run, chamber and recipe. The prototype data model explicitly models these relationships rather than treating the fab as one flat table.

3. Spatial defects can carry process information

A wafer is a spatial object. Defect locations can form center-heavy, edge-ring, localized-hotspot, radial or elongated scratch-like patterns. LogicX classifies these signatures with explicit geometric heuristics, then provides the resulting pattern score as one evidence stream inside root-cause fusion. The repository explicitly labels these heuristics as demonstration methods rather than production-certified defect classifiers. [3]

4. Prevention must not leak future information

A pre-run predictor that accidentally reads future yield, future inspection data or future maintenance would produce a misleadingly strong model. LogicX therefore defines `planned_start_at` as a hard cutoff and rejects forbidden future features at training and inference time. [5]

Solution Overview

LogicX as a semiconductor analytics control tower

LogicX is designed for process and yield engineers who need one analytical surface for fleet health, deep lot investigations, wafer pattern review, root-cause evidence and pre-run risk. The frontend is a dashboard rather than a conversation interface; the backend owns the analytics; the database preserves lineage and audit objects.

Functional modules

Module	Purpose	Key outputs
Monitor	Fleet-level operational health and yield excursions	fleet summary, excursion indicators, recurrence signals
Investigate	Trace a selected lot into its process context	parameter anomalies, chamber evidence, similar lots
Wafer Pattern Lab	Explain spatial defect signatures	pattern class, score, wafer-level distribution
Pre-Run Risk	Identify future lots that warrant attention before processing	risk_score, risk_class, top features
Evidence	Make every candidate auditable	supporting signals, contradicting signals, evidence IDs
Actions / Review	Turn recommendations into governed engineering workflow	pending -> acknowledged -> approved/rejected -> completed

Prototype UI Coverage - reconstructed from repository screen inventory

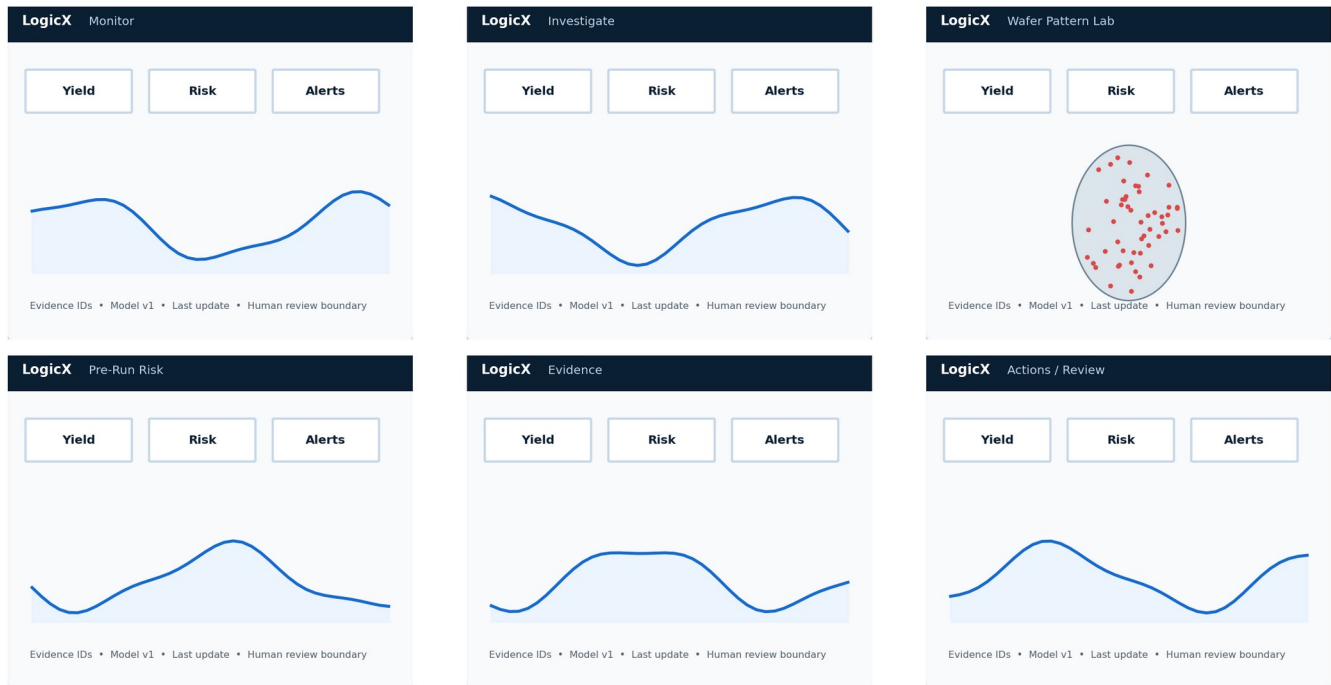
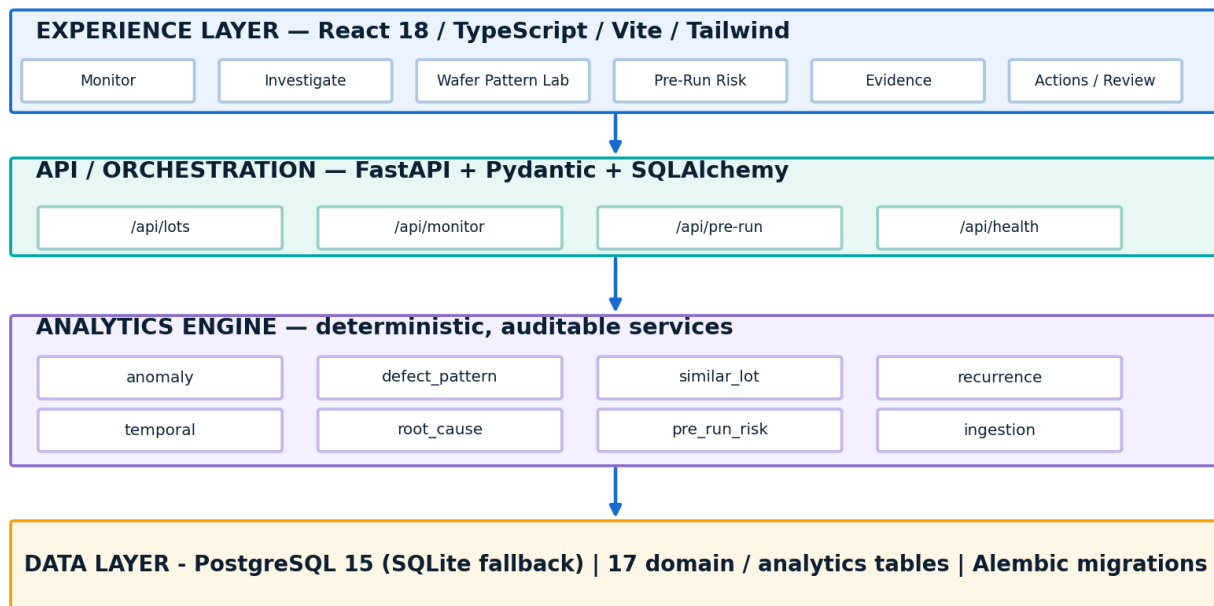


Figure 3. Reconstructed screen inventory based on the repository: Monitor, Investigate, Wafer Pattern Lab, Pre-Run Risk, Evidence and Actions / Review.

Technical Architecture

Browser -> API -> analytics -> database

LogicX System Architecture



Prototype: single-process analytics control tower; no message queue or microservice mesh in the current version.

Figure 4. LogicX technical architecture.

The current repository describes the system as a single-process analytics control tower: React frontend, FastAPI backend, SQLAlchemy ORM and PostgreSQL 15, with SQLite available as a lightweight fallback. There is no message queue or microservice mesh in the prototype. [2]

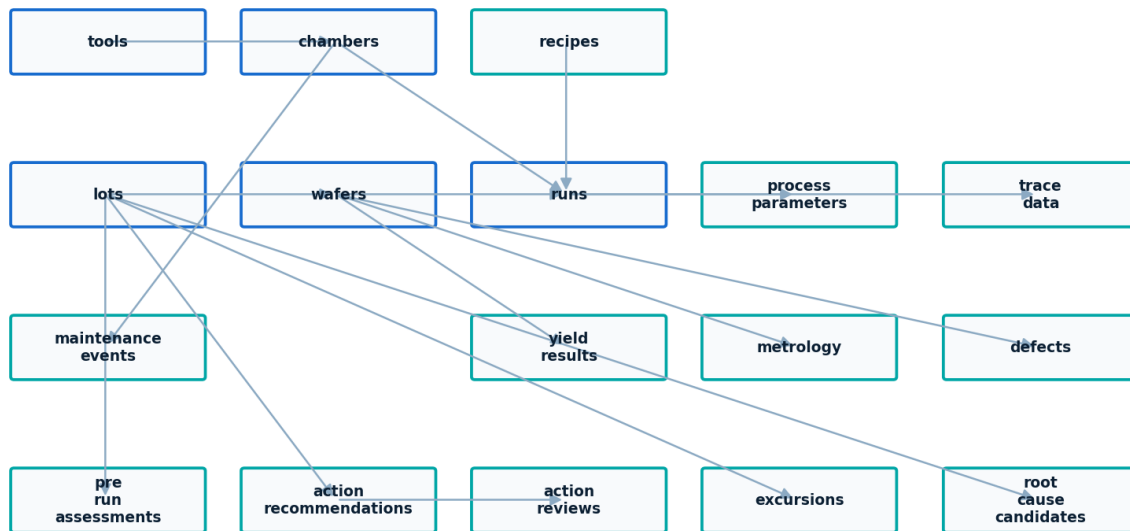
Technology stack

Layer	Technologies	Role
Frontend	React 18, TypeScript, Vite, Tailwind CSS	Screen rendering, navigation, typed API contracts
API	Python, FastAPI, Pydantic	HTTP orchestration and typed JSON responses
Data access	SQLAlchemy ORM	Database identity and query layer
Analytics	pandas, NumPy, SciPy, scikit-learn	Numerical analysis, spatial logic, similarity and model training
Database	PostgreSQL 15; SQLite fallback	Persistent relational store
Migrations	Alembic	Schema evolution
Testing	pytest; TypeScript compiler	Regression, guardrail and type checks
Containerization	Docker, Docker Compose	Reproducible local full-stack deployment

Data Architecture and Domain Model

The fab genealogy represented in relational form

Relational Domain Model (17 Tables)



Business keys such as lot_id and chamber_id remain separate from integer primary keys; outputs preserve evidence lineage.

Figure 5. Simplified entity relationship map for the 17-table prototype schema.

The repository documents 17 ORM tables that represent the genealogy from physical tool through chamber, run, wafer and lot into process parameters, trace data, metrology, defects and yield. Separate analytics output entities store excursions, root-cause candidates, pre-run assessments and action review state. [6]

Core identity chain

```

tool -> chamber -> run -> wafer -> lot
      |
      +-> process_parameters
      +-> trace_data
      +-> metrology
      +-> defects
      +-> yield_results
chamber -> maintenance_events
lot -> excursions / root_cause_candidates / pre_run_assessments / actions

```

Why this matters in a real fab

A production deployment would need the same logical chain even if the physical storage technologies changed. For example, MES may own lot state, a historian may own sensor traces, an inspection system may own defect records, and a CMMS may own maintenance. LogicX can then sit as an analytical layer over canonical identities and time-aligned joins.

End-to-End Data Flow

How one lot moves through the analytical pipeline

End-to-End Data Flow

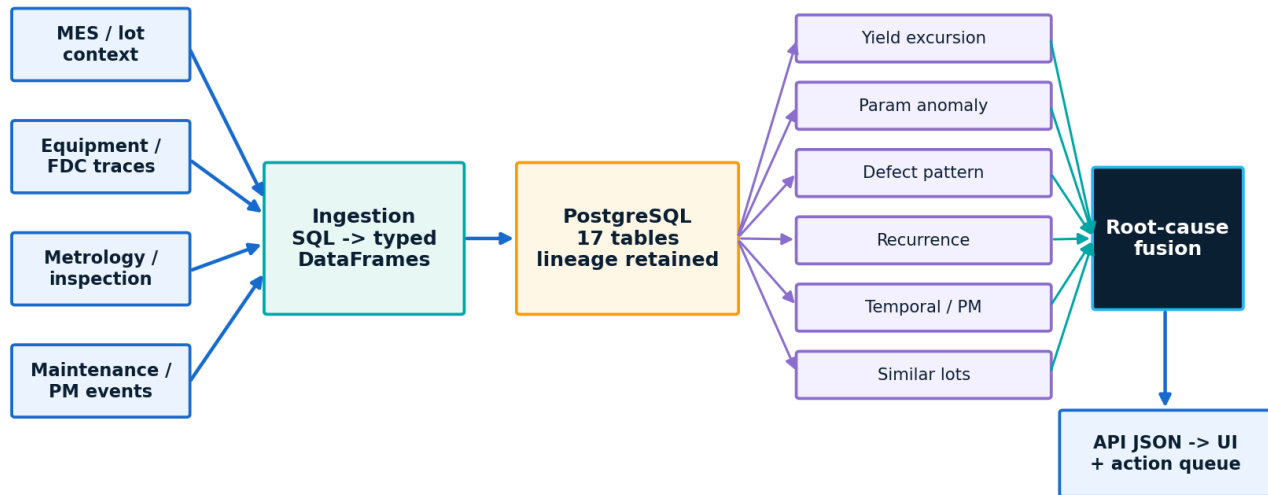


Figure 6. Data movement from source evidence through analytics and back to the engineer.

Request lifecycle

- 1. A target lot is selected from the UI.
- 2. The backend ingestion layer pulls the relevant historical DataFrames from the database.
- 3. Yield excursion detection establishes whether the lot is outside its robust historical baseline.
- 4. Process parameter analysis identifies out-of-control or strongly deviating parameters.
- 5. Wafer defect coordinates are classified into spatial signatures.
- 6. Chamber recurrence and maintenance proximity add equipment-history evidence.
- 7. Historical similarity returns known-outcome analogues.
- 8. Root-cause fusion combines the available evidence into controlled-vocabulary candidates.
- 9. The API returns typed JSON including evidence IDs, model version and timestamp.
- 10. The UI renders the candidate set and routes any recommendation into a human review state.

Analytics Engine I - Yield Excursion and Parameter Anomalies

Robust baselines instead of fragile single-threshold alerts

Yield excursion detection

The prototype uses Median Absolute Deviation (MAD) because a mean and standard deviation can be distorted by extreme excursions. It computes the historical median yield, the median absolute deviation, and a scaled deviation score. A minimum history of 10 lots is required; otherwise the result is insufficient_data. [3]

```
baseline_yield = median(historical_yields)
MAD = median(abs(historical_yields - baseline_yield))
deviation_sigma = (observed_yield - baseline_yield) / (1.4826 * MAD)

|sigma| > 2.5  -> mild
|sigma| > 3.5  -> moderate
|sigma| > 5.0  -> severe
```

Parameter anomaly detection

For each process parameter present in the target lot, the prototype calculates a z-score against the historical population for that parameter, excluding the lot under investigation. Flags begin at absolute z-score above 3.0, with higher severity bands above 4.5 and 6.0. [3]

In production, the same mathematical pattern can be retained while replacing demonstration thresholds with statistically validated control limits, recipe-family baselines, chamber-specific distributions, and production qualification rules.

Analytics Engine II - Wafer Spatial Defect Patterns

Turning defect coordinates into a process signal

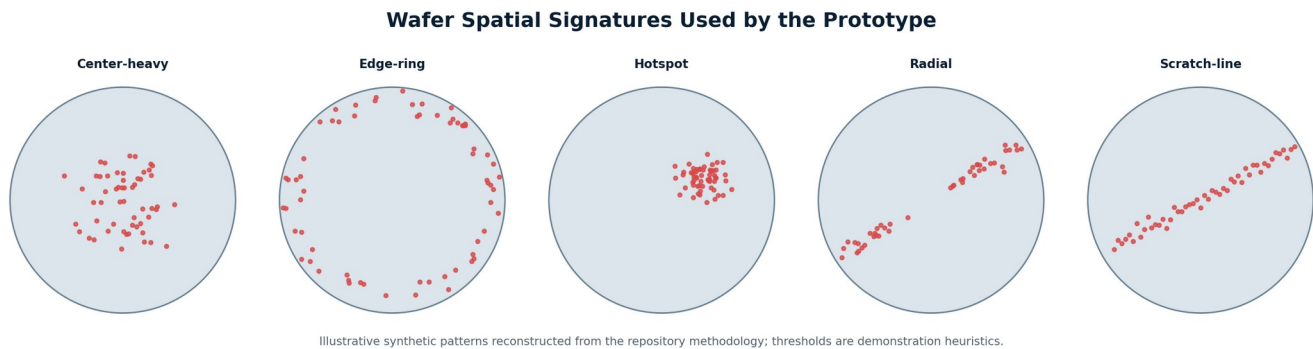


Figure 7. Illustrative spatial signatures used by the current prototype.

Pattern	Prototype rule	Engineering interpretation to investigate
Center-heavy	>40% defects within 0.4 x wafer radius	Potential center-localized process or handling signature; requires domain review
Edge-ring	>40% beyond 0.8 x radius	Edge-related process, bevel, coating or handling mechanisms may merit review
Localized hotspot	One compact cluster contains >50% of defects	Local equipment, pattern or handling event
Radial	Angular entropy < 1.5 bits	Directional / sector-specific process signature
Scratch-line	PCA aspect ratio > 5:1	Elongated defect cloud consistent with a line-like mechanism
Uniform	None of the above	No dominant spatial signature under prototype heuristics

The important design decision is that spatial classification is not treated as a standalone root-cause answer. It becomes one weighted input to evidence fusion. That avoids over-interpreting a single image or wafer map.

Analytics Engine III - Chamber Recurrence, Maintenance and Temporal Precedence

Connecting yield shifts to equipment history

Chamber recurrence

For each chamber, the prototype compares mean chamber yield to the fleet mean and computes a normalized recurrence score. A chamber is flagged when its yield gap exceeds four percentage points and at least five lots have used it. The normalization uses a hyperbolic tangent so one extreme event does not saturate the score. [3]

```
yield_gap = fleet_mean - chamber_mean
recurrence_score = tanh((yield_gap / fleet_std) * 2)
flag if yield_gap > 0.04 and chamber_lot_count >= 5
```

Maintenance proximity

Maintenance events are evaluated within a +/-14 day window. The prototype compares post-maintenance yield behavior with a pre-maintenance chamber baseline and derives a shift score. The lot is marked with temporal_precedence when the maintenance event occurred before the lot start.

Why temporal precedence matters

Correlation is weaker when the suspected cause happens after the effect. LogicX therefore records whether the candidate event preceded the yield drop and exposes this as structured evidence rather than narrative speculation.

Analytics Engine IV - Similar-Lot Historical Matching

Using analogy as supporting evidence

The similar-lot service represents each historical lot using normalized yield, mean parameter deviation, defect density and primary-chamber yield gap. StandardScaler normalization is followed by cosine similarity, and the top five historical analogues are returned with their known outcomes. [3]

Engineering purpose

Similar-lot retrieval is a memory mechanism for the analytics system. It does not prove causality. Its role is to answer: "Have we seen a comparable combination of conditions before, and what happened then?"

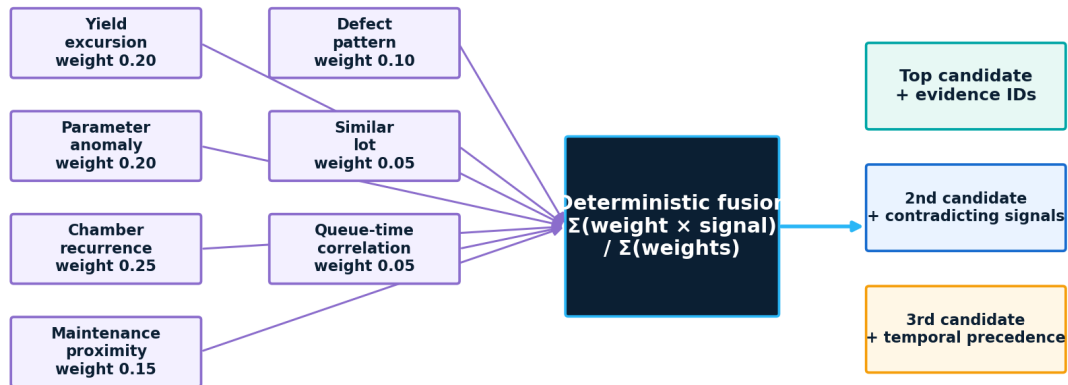
Example evidence chain

Evidence item	Value for engineer
Target lot	Observed yield excursion and current context
Similar lot A	Comparable parameter deviation and chamber exposure; prior yield outcome known
Similar lot B	Same chamber family, different recipe; outcome helps separate process from tool effects
Similar lot C	Similar defect density but different spatial pattern; may reduce confidence in one hypothesis

Root-Cause Evidence Fusion

Deterministic ranking of hypotheses, not confirmation

Seven-Signal Evidence Fusion



Root-cause types are controlled vocabulary entries; the prototype explicitly avoids LLM inference in the scoring path.

Figure 8. Seven evidence signals flow into a fixed, inspectable fusion function.

The repository defines seven signals: yield excursion, parameter anomaly, chamber recurrence, maintenance proximity, defect pattern, similar-lot evidence and queue-time correlation. Each cause candidate receives the signals that apply to it; the signals are combined with hard-coded weights. [4]

Signal	Default weight	What it measures
yield_excursion	0.20	How far the lot yield moved from its robust baseline
param_anomaly	0.20	Strength of process parameter deviation
chamber_recurrence	0.25	Systematic chamber yield deficit
maintenance_proximity	0.15	Evidence of post-PM instability and temporal precedence
defect_pattern	0.10	Strength of spatial defect signature
similar_lot	0.05	Outcome of similar historical lots
queue_time	0.05	Exposure-time correlation

```
final_score = sum(weight_i * signal_i) / sum(weight_i)
confidence = min(1.0, (evidence_count / 4) * final_score)
return top 3 candidates by final_score
```

Controlled cause vocabulary

Candidates are limited to etch_drift, overlay_excursion, particle_contamination, maintenance_shift, chamber_recurrence, temporal_exposure and unknown. New cause types require code changes rather than dynamic text generation. [4]

Interpretation rule: The system returns the highest-ranked candidates based on available evidence. It does not claim causal confirmation and does not replace engineering judgment. This wording is a deliberate governance boundary in the prototype.

Pre-Run Risk Prediction

Flagging upcoming lots before processing starts

Pre-Run Risk: Explicit No-Future-Data-Leakage Boundary

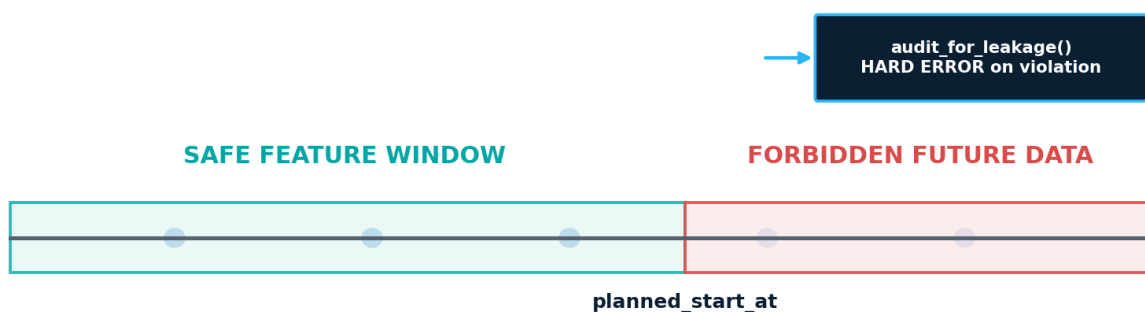


Figure 9. Explicit `planned_start_at` cutoff prevents future-data leakage.

The pre-run risk model exists for a different decision: not "why did this lot fail?" but "does this upcoming lot resemble historical conditions associated with low yield?" The model is allowed to use lot creation attributes and historical chamber/equipment features available before the planned start time. [5]

Feature whitelist

Allowed feature	Meaning
<code>lot_priority</code>	0=low, 1=normal, 2=high
<code>product_encoded</code>	Product family encoding
<code>technology_node_encoded</code>	Node / technology encoding
<code>planned_recipe_encoded</code>	Recipe from lot plan
<code>planned_chamber_encoded</code>	Assigned chamber
<code>chamber_yield_30d</code>	Trailing 30-day chamber mean yield
<code>days_since_last_pm</code>	Days since last PM
<code>chamber_ooc_rate_30d</code>	Trailing 30-day out-of-control rate
<code>queue_time_planned_h</code>	Planned queue time in hours

The prototype uses `GradientBoostingClassifier` with a chronological holdout of the most recent 20% for performance reporting. Risk is returned as a probability plus low / medium / high class. The repository also documents prototype limitations: no cross-validation, no hyperparameter tuning, retraining on each request, approximated rolling features and class imbalance. [5]

Governance, Auditability and Human Approval

Designing for safe engineering use

Safety, Auditability and Human Governance

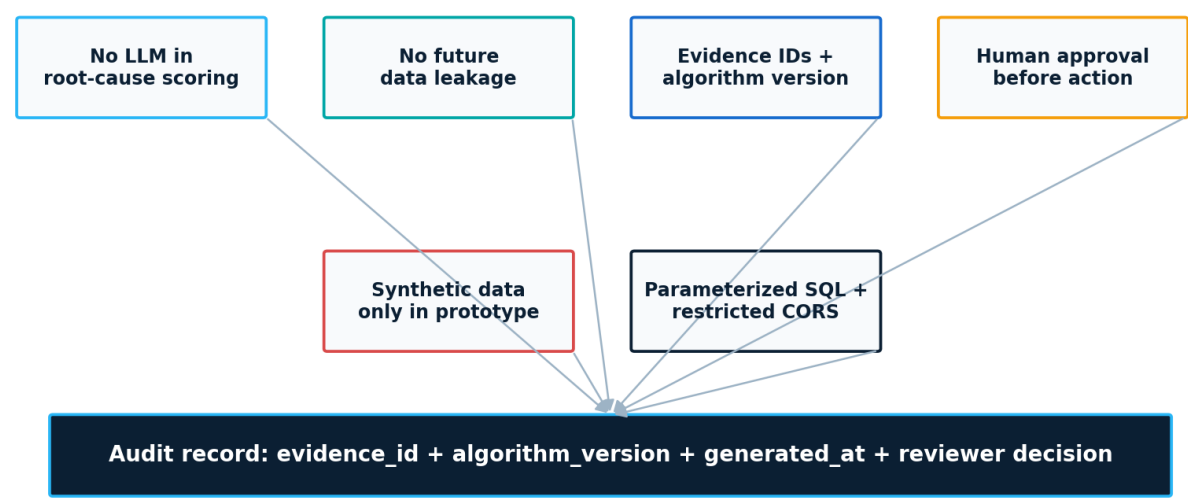


Figure 10. Governance boundaries implemented in the prototype.

Five architectural guardrails

Guardrail	Implementation
Zero LLM core inference	Root-cause scores are computed only by deterministic services; static tests prevent LLM imports in the root-cause module.
No future data leakage	Hard feature whitelist + audit_for_leakage() at training and inference.
Absolute traceability	evidence_id + algorithm_version + generated_at on analytical outputs.
Human-in-the-loop	Recommendations remain pending until a named reviewer approves or rejects them.
Isolated data space	Synthetic data only; no production PII or real fab IP in the demonstration environment.

Action lifecycle

pending -> acknowledged -> approved | rejected -> completed

No corrective action is executed automatically. This is essential because a root-cause hypothesis can be wrong even when its evidence score is high. The current design keeps the action boundary with the engineer and captures the decision.

Prototype UI and Operator Experience

How the dashboard expresses the analytics

Prototype UI Coverage - reconstructed from repository screen inventory

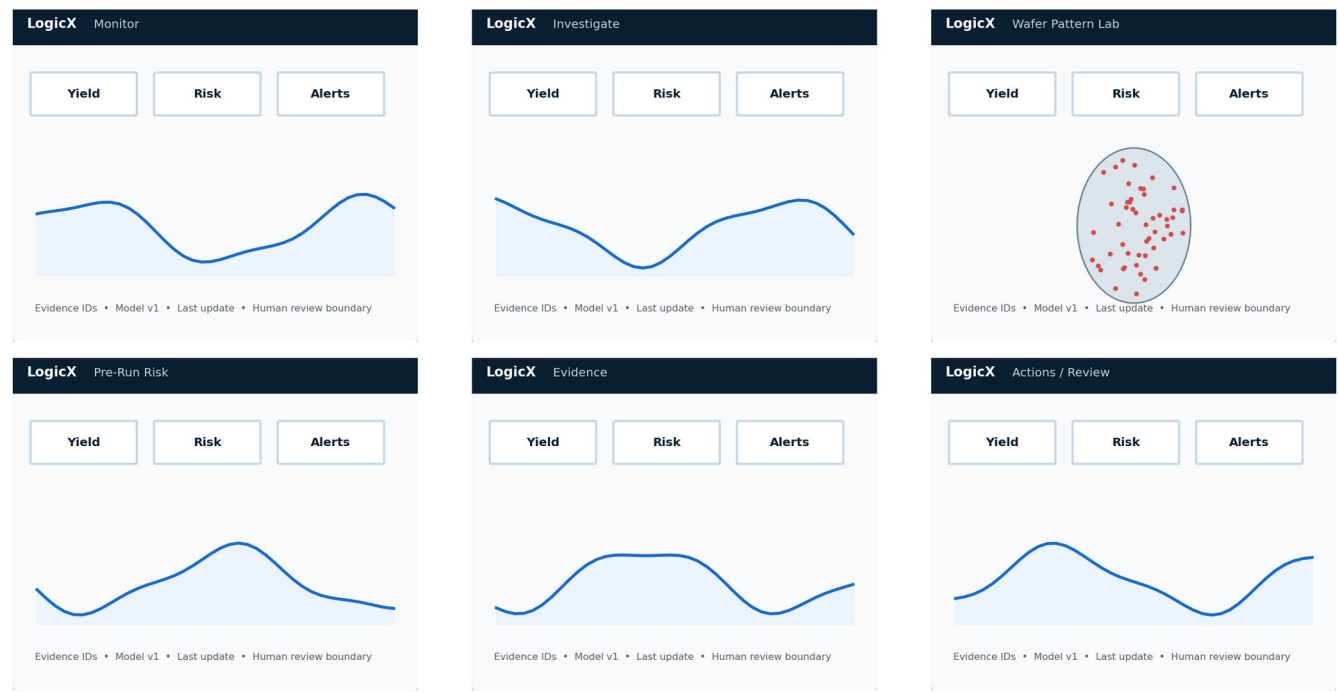


Figure 11. Screen coverage reconstructed from the repository screen inventory.

Screen	Primary question answered
Monitor	Is the fleet healthy, and where is the current excursion?
Investigate	What changed on this lot, and which parameters / chambers are unusual?
Wafer Pattern Lab	What spatial defect signature is present across the wafers?
Pre-Run Risk	Which upcoming lots warrant enhanced review before the first process step?
Evidence	Why did the system produce these candidates, and what records support them?
Actions / Review	What should the engineer acknowledge, approve, reject or track to completion?

The frontend is intentionally thin. Analytics logic belongs in backend services so the UI cannot silently diverge from the mathematics, and typed API schemas provide a stable interface between the analytical engine and the operator experience.

End-to-End Synthetic Demonstration Case

A judge walkthrough using the prototype's logic

Case status

The following numbers are illustrative demonstration values, not real fab measurements. They are included to explain the workflow and should not be read as production claims.

Stage	Illustrative observation	What LogicX does
Fleet monitor	Target lot yield is materially below recent baseline	Runs robust excursion detection
Process context	Two process parameters show unusually large z-scores	Flags parameter anomalies and severity
Wafer map	Defects form an elongated line-like pattern	Classifies scratch_line pattern
Equipment history	Target lot used a chamber with recurring yield deficit	Adds chamber recurrence evidence
Maintenance history	Recent PM occurred before the lot	Adds maintenance proximity + precedence
Historical analogy	Several similar lots had poor outcomes	Adds similarity evidence
Fusion	Multiple independent signals support a small set of hypotheses	Ranks controlled cause candidates
Action	Engineer reviews evidence	Recommendation remains pending until human approval

Why this is more than an alert

An alert tells the engineer that something is wrong. LogicX packages the surrounding evidence into one investigation path. This is the difference between a monitoring dashboard and an engineering control tower: the output is not just a red tile; it is a structured evidence bundle with provenance and a next-step workflow.

Worked Example of Root-Cause Scoring

Showing the mathematics transparently

Assume a synthetic case has the following normalized signal values for one candidate. The values below are invented solely to demonstrate the calculation mechanics.

Signal	Weight	Synthetic signal score	Contribution
yield_excursion	0.20	0.90	0.180
param_anomaly	0.20	0.80	0.160
chamber_recurrence	0.25	0.85	0.213
maintenance_proximity	0.15	0.70	0.105
defect_pattern	0.10	0.60	0.060
similar_lot	0.05	0.75	0.038
queue_time	0.05	0.40	0.020
Total	1.00		0.776

Using the prototype formula, the illustrative final score would be approximately 0.776. If four independent signals materially contribute, the prototype confidence transformation would be bounded at $\min(1.0, (4/4) \cdot 0.776) = 0.776$. This is a demonstration of the formula, not a validated confidence interval or probability of causality.

Judge takeaway

The important property is not the sample score. It is that an engineer can recompute the score, inspect every input signal, view supporting evidence IDs, and see the algorithm version that generated the result.

Testing and Quality Controls

What is already protected in the prototype

The repository describes 100 backend unit and regression tests plus a TypeScript compiler check for frontend type consistency. Tests cover not only normal analytics behavior but also the safety rules that make the architecture auditable. [1][2]

Test class	Purpose
Analytics regression	Protect expected behavior of excursion, anomaly, pattern, recurrence and similarity functions.
Leakage audit	Fail hard when forbidden post-start features enter the pre-run model.
No-LLM guardrail	Prevent accidental LLM imports in the deterministic root-cause path.
Schema / API typing	Keep response shapes aligned between FastAPI and React.
Data / seed reproducibility	Maintain deterministic synthetic behavior for demonstrations.
Human approval state	Ensure action recommendations cannot bypass the review lifecycle.

Recommended production test expansion

- Replay a library of historical excursions with frozen expected evidence bundles.
- Back-test pre-run risk chronologically and report precision, recall, calibration and operational cost of false alerts.
- Stress-test time alignment, missing data, delayed inspection events and chamber reassignment.
- Add contract tests for every production connector and schema version.
- Measure model / rule drift and create explicit recalibration gates before changing thresholds or weights.

Deployment and Setup

How the prototype is intended to run

The repository provides both native Windows development instructions and Docker Compose instructions. The local frontend runs on port 5173 and the FastAPI backend on port 8000; PostgreSQL is used in the Docker path and SQLite is available for lightweight local development. [2]

```
# Full-stack Docker path
docker compose up --build -d

# Native backend
cd src/backend
pip install -e "[dev]"
alembic upgrade head
python data/seed_db.py
uvicorn app.main:app --reload

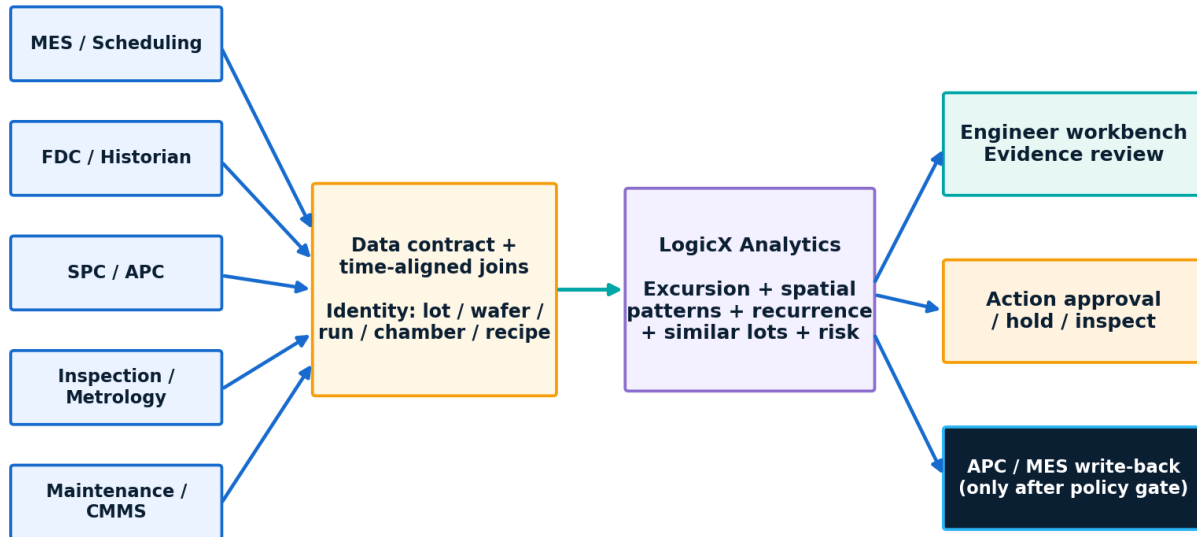
# Frontend
cd src/frontend
npm ci
npm run dev
```

Runtime surface	Prototype endpoint / location
Dashboard	http://127.0.0.1:5173
FastAPI	http://127.0.0.1:8000
OpenAPI docs	http://127.0.0.1:8000/api/docs
Health check	/api/health
Fleet summary	/api/monitor/fleet-summary

Production Integration Architecture

How LogicX would connect to a real semiconductor factory

Production Integration Path: Prototype -> Fab



The current prototype stops at recommendation and human approval; production write-back is a future integration step.

Figure 12. Production integration path mapped from prototype concepts to common fab system boundaries.

Canonical integration contracts

Prototype entity	Likely production source	Required contract
lots / wafers	MES / lot tracking	Stable lot ID, wafer ID, product, node, timestamps
runs / chambers / recipes	MES + tool integration	Run start/end, chamber, recipe and status
process_parameters	FDC / historian	Parameter name, value, unit, nominal, limits, timestamp
trace_data	FDC / historian	Channel identity, timestamp, value, sampling metadata
metrology	Metrology system	Metric, value, unit, wafer and measurement site
defects	Inspection system	Wafer coordinates, class, inspection timestamp, confidence
maintenance_events	CMMS / EAM	Chamber, event type, performed_at, status
yield_results	E-test / yield system	Wafer yield, fail bins, test timestamp

The production system should not be built as a set of ad-hoc database joins. It should have a canonical data contract with identity, timestamp semantics, units, late-arriving data rules, missing-data handling and versioned schemas. The prototype's relational model provides the starting conceptual map for that contract.

Scalability and Production Engineering

What changes when synthetic data becomes a live fab

Concern	Prototype	Production direction
Data volume	735K generated points in a deterministic demo dataset	Partitioned time-series storage, retention policies and materialized aggregates
Latency	On-request analytics	Precompute fleet metrics; async execution for deep investigations
Multi-fab	Single-fab context	Tenant / fab key in all relevant entities + row-level security
Model training	Pre-run model retrains on request	Scheduled training + model registry + validation gates
Feature computation	Some rolling features approximated	Reliable offline/online feature pipelines
Audit	Database-level provenance fields	Immutable audit store and change-controlled model/rule registry
Availability	Local prototype	HA deployment, observability, backups and disaster recovery

Scalability principle

The analytical contract should remain stable while implementation changes underneath it. Engineers should continue receiving the same concepts - excursion, evidence, candidate cause, risk, reviewer state - even as storage and execution become distributed.

Real-Industry Equipment Mapping

How the prototype concept aligns with actual semiconductor inspection and metrology workflows

Industry equipment vendors publicly describe optical metrology, e-beam inspection, wafer defect inspection and integrated data analysis as parts of semiconductor production control. ASML documents YieldStar optical metrology and HMI e-beam inspection, while KLA describes optical and e-beam inspection systems, defect classification, process debug and process-window monitoring. [7][8][9]

What those systems contribute to a LogicX-style control tower

Real-industry capability	Contribution to LogicX data model
Optical metrology	Pattern / overlay / CD measurements for lot and wafer context
E-beam inspection	High-resolution defect observations and hotspot analysis
Optical wafer inspection	High-throughput defect discovery and classification
FDC / sensor monitoring	Process trace and parameter anomaly evidence
MES / scheduling	Lot genealogy, process plan and planned chamber / recipe
Maintenance / qualification	Temporal evidence for post-PM instability

Visual reference note

The explanatory figures in this dossier are original engineering diagrams rather than copied vendor photography. The cited industry pages identify the real equipment categories and workflows that motivate the integration mapping.

Prototype Limitations and Validation Requirements

Where the demo must not be over-interpreted

Limitation	Why it matters	Required production validation
Synthetic data	Does not capture real process noise, tool-specific fingerprints or fab IP	Train / calibrate on governed historical fab data
Heuristic spatial classifier	Not production-certified	Validate against expert-labeled inspection data
Fixed fusion weights	Weights are hand-set constants	Tune or redesign with historical RCA outcomes and domain review
No causal inference	Correlation and precedence are not proof	Use controlled experiments / engineering confirmation
Pre-run model constraints	Some rolling features are approximated in the demo	Implement exact time-aware feature generation
No authentication in prototype	Suitable only for controlled demo use	Integrate SSO, RBAC, service identity and secrets management
Prototype write-back stops at approval	No direct shop-floor automation	Add policy-controlled APC/MES integration only after validation
Model / rule drift	Fab processes evolve	Version every rule/model and schedule validation

Success Metrics and Evaluation Plan

How to prove value after integration

Production Validation KPI Framework



No production benefit is claimed by the prototype. These measures define the validation plan required after real fab data onboarding.

Figure 13. Production validation KPI framework.

The prototype does not claim a measured production yield uplift. Instead, it defines the metrics a real pilot should evaluate. The most important evaluation principle is to measure the complete workflow, not just model accuracy: did the system reduce time to a credible engineering hypothesis, did it improve evidence completeness, and did approved interventions change the outcome?

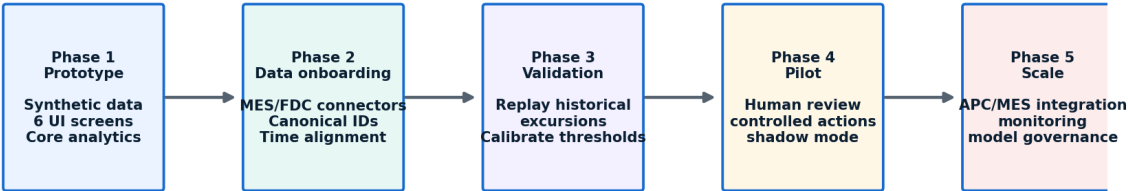
Suggested pilot design

- Select a bounded production area and a fixed historical window.
- Create a frozen set of known excursions and evaluate LogicX in replay mode.
- Run shadow mode before any action is allowed to influence production.
- Compare investigation cycle time and evidence quality with the existing engineering workflow.
- Measure false alerts and missed excursions, not only successful detections.
- Validate the pre-run risk model strictly chronologically to avoid leakage.
- Only after governance review, introduce a controlled recommendation-to-action loop with rollback and audit.

Roadmap to a Real-Fab Product

From hackathon prototype to validated engineering platform

Implementation Roadmap



Key engineering principle: validate against known excursions and preserve the approval boundary while increasing automation around evidence collection.

Figure 14. Proposed roadmap from prototype to production scale.

Phase	Primary deliverable	Exit criterion
1. Prototype hardening	Stable demo, tests, documentation, evidence bundles	Repeatable demo and regression suite
2. Data onboarding	Canonical lot/wafer/run/chamber identity and connectors	Time-aligned data contract passes replay checks
3. Historical validation	Thresholds, defect labels, fusion calibration	Known-excursion replay shows measured performance
4. Pilot / shadow mode	Engineer-facing recommendations with no automatic write-back	Reviewers trust evidence and alert burden is acceptable
5. Controlled operations	Approved actions integrated through policy boundary	Every action has approval, rollback and audit
6. Multi-fab scale	Tenant isolation, HA, observability and model governance	Operational SLOs and change controls established

Judge Q&A - Strong Technical Answers

Questions a semiconductor / AI / software judge is likely to ask

Q: Is this just a chatbot?

A: No. The critical path is a deterministic analytics pipeline. The UI is a dashboard, root-cause candidates come from explicit algorithms, and evidence is stored structurally.

Q: Why not use an LLM for root-cause analysis?

A: Because the prototype prioritizes auditability and repeatability for an engineering decision path. A language model could assist with summarization outside the scoring core, but it is intentionally excluded from the ranking function.

Q: Does it prove the root cause?

A: No. It ranks the highest-supported candidates using the available evidence and explicitly requires engineering review.

Q: How do you prevent data leakage in pre-run prediction?

A: `planned_start_at` is the hard cutoff. A whitelist defines the only legal features; `audit_for_leakage` rejects forbidden columns at both training and inference.

Q: What data do you need from a real fab?

A: At minimum: lot/wafer genealogy, run/chamber/recipe history, process parameters and traces, metrology/inspection outputs, maintenance events and electrical yield outcomes.

Q: Can it scale?

A: Yes conceptually, but the prototype is not the production deployment. The documented path is to move from on-request analysis to precomputed aggregates, time-series partitions, scheduled model training, multi-fab isolation and HA.

Q: What is the unique value?

A: Evidence fusion across multiple engineering views: yield excursion, parameter anomaly, wafer spatial pattern, chamber recurrence, maintenance timing, historical similarity and queue-time context, presented in one reviewable workflow.

Q: What is the main limitation?

A: The current data is synthetic and the thresholds / weights are demonstration parameters. Real fab validation is essential before operational decisions.

Appendix A - Prototype File and Service Map

How the repository is organized

Path / module	Purpose
src/backend/app/models/	17 ORM domain / analytics entities
src/backend/app/services/anomaly.py	Yield excursion and process-parameter anomaly logic
src/backend/app/services/defect_pattern.py	Wafer spatial signature classification
src/backend/app/services/similar_lot.py	Cosine-similarity historical matching
src/backend/app/services/recurrence.py	Chamber recurrence scoring
src/backend/app/services/temporal.py	Maintenance proximity and precedence
src/backend/app/services/root_cause.py	Deterministic evidence fusion and Top-3 candidates
src/backend/app/services/pre_run_risk.py	Whitelisted pre-run risk model + leakage audit
src/backend/data/synthetic/	Deterministic synthetic data generation
src/frontend/src/screens/	Six main dashboard views
src/frontend/src/api/	Typed API client infrastructure
src/backend/tests/	Regression and safety tests
docs/	Deep technical methodology and setup documents

Appendix B - Semiconductor and Analytics Glossary

Terms that make the prototype easier to evaluate

Term	Meaning in this dossier
Fab	Semiconductor fabrication facility.
Lot	Primary production / scheduling unit containing a set of wafers.
Wafer	Silicon substrate processed through repeated manufacturing steps.
Chamber	A processing sub-unit inside a semiconductor tool.
Recipe	Configured process recipe used by a tool / chamber.
Yield	Fraction of dies or wafers meeting required electrical / process outcomes.
Metrology	Measurement of physical / process properties such as CD, overlay or film thickness.
Inspection	Detection / classification of physical or pattern defects.
FDC	Fault Detection and Classification; time-series process/equipment monitoring.
PM	Preventive maintenance.
MAD	Median Absolute Deviation, a robust measure of dispersion.
Z-score	Standardized deviation from a baseline mean in units of standard deviation.
Spatial signature	Geometric pattern formed by defect coordinates on a wafer.
Evidence fusion	Combining multiple independent signals into a single auditable score.
Pre-run	Before the planned start of a lot; no future operational data is allowed.
Root-cause candidate	A ranked hypothesis that engineers must review; not proof of causality.

Appendix C - Sources and Reference Material

Primary implementation sources and industry context

The repository and its technical documents are the implementation sources of truth. Industry sources are used only to establish manufacturing context and explain why the prototype data model and workflow map to real semiconductor operations.

- [1] **LogicX repository README** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX> - architecture, stack, guardrails, synthetic-data boundary
- [2] **LogicX architecture document** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX/blob/main/docs/architecture.md> - system architecture, data flow, auditability, action approval
- [3] **LogicX analytics methodology** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX/blob/main/docs/analytics-methodology.md> - MAD, z-score, spatial patterns, recurrence, temporal and similarity methods
- [4] **LogicX root-cause methodology** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX/blob/main/docs/root-cause-methodology.md> - seven-signal fusion, controlled cause vocabulary, output schema
- [5] **LogicX pre-run risk** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX/blob/main/docs/pre-run-risk.md> - feature whitelist, leakage enforcement, model behavior and limitations
- [6] **LogicX data model** - <https://github.com/ronaksavsani181/bob-ai-hackathon-LogicX/blob/main/docs/data-model.md> - 17-table domain model and field definitions
- [7] **ASML - Measuring accuracy / lithography principles** - <https://www.asml.com/technology/lithography-principles/measuring-accuracy> - metrology, e-beam inspection, sensors and data analysis context
- [8] **ASML - Wafer metrology and inspection systems** - <https://www.asml.com/products/metrology-and-inspection-systems> - YieldStar and HMI inspection context
- [9] **KLA optical inspection machine series description** - <https://www.directindustry.com/prod/kla-corporation/product-113449-1102971.html> - industry context for wafer defect inspection, classification and monitoring
- [10] **Semiconductor Engineering - wafer edge inspection and yield analysis** - <https://semiengineering.com/criticality-of-wafer-edge-inspection-and-metrology-data-to-all-surface-defectivity-root-cause-and-yield-analysis/> - industry discussion of inspection data for edge-yield root-cause work

Final Judge Summary

A one-page mental model of LogicX

Problem

Yield loss in semiconductor manufacturing is a multi-source, time-dependent engineering problem. Detecting the drop is not enough; the engineer needs a structured route from excursion -> evidence -> hypothesis -> action.

Product

LogicX is a manufacturing analytics control tower that combines fleet monitoring, lot investigation, wafer spatial analysis, chamber recurrence, historical similarity, pre-run risk and governed actions.

Technical differentiator

The prototype keeps the decision core deterministic, auditable and explicit: seven evidence signals, controlled cause vocabulary, evidence IDs, model versions and no automatic shop-floor action.

Industry bridge

The same concepts align with real semiconductor workflows that already generate process, metrology, inspection and equipment data. The production challenge is integration, validation and governance - not a rewrite of the core engineering idea.

Honest boundary

This is a synthetic-data prototype, not a validated fab system. The next stage is historical replay and shadow-mode validation with governed production data before any operational write-back.

Recommended judge demo sequence: Monitor -> Investigate -> Wafer Pattern Lab -> Evidence / Root Cause -> Pre-Run Risk -> Actions / Review. End by showing the explicit human-approval boundary and the synthetic-data disclaimer.